

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Автономова Ксения

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В лабораторной работе №3 мне нужно было перенести ранее разработанное приложение на фреймворк Vue.JS.

В качестве основы я использовала проект по варианту «Платформа для покупки билетов на мероприятия». В первой лабораторной работе был создан основной интерфейс сайта, во второй лабораторной работе он был подключён к mock API, а после выполнения ДЗ4 в проекте появились SVG-иконки через общий SVG-спрайт.

В данной лабораторной работе нужно было сохранить внешний вид и функциональность проекта, но изменить его внутреннюю структуру. Теперь приложение должно работать как SPA на Vue.JS.

По заданию нужно было реализовать:

- подключение роутера;
- работу с внешним API через axios;
- разумное деление на компоненты;
- использование composable для повторяющейся логики.

Ход работы

1. Общая идея работы

Я продолжила работать с темой платформы для покупки билетов на мероприятия в Санкт-Петербурге.

В предыдущих версиях проекта уже были реализованы основные страницы:

- главная страница;
- вход;
- регистрация;
- поиск мероприятий;
- страница мероприятия;
- личный кабинет пользователя;
- кабинет организатора.

В лабораторной работе №3 эти страницы были перенесены во Vue-приложение. Внешний вид сайта остался похожим на предыдущую версию, но теперь страницы являются не отдельными HTML-файлами, а Vue-компонентами.

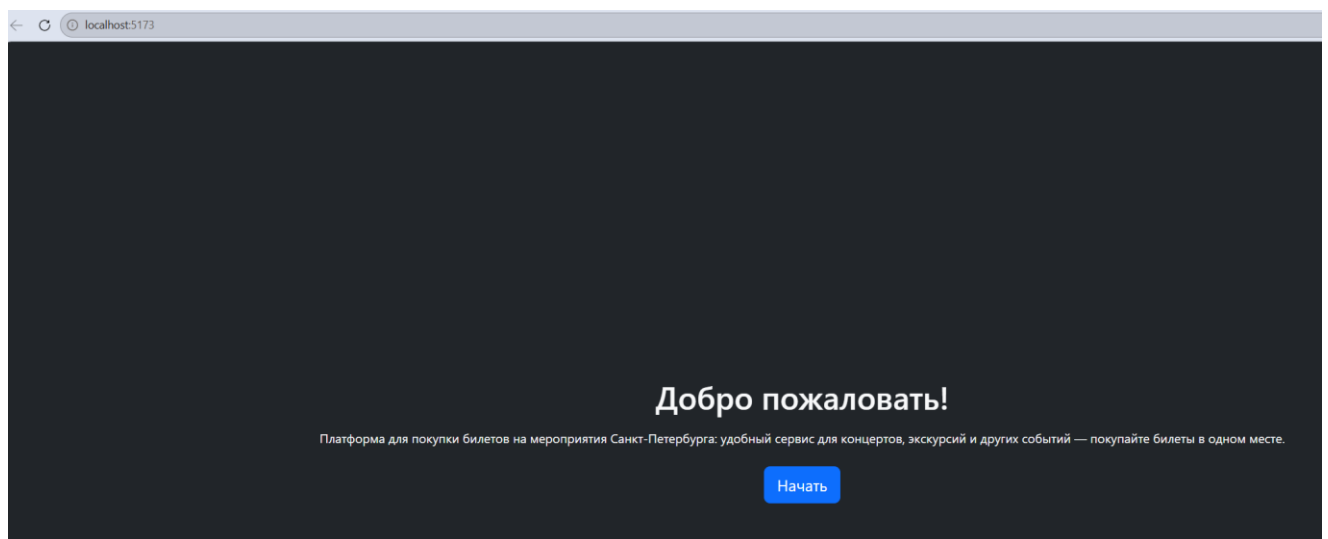


Рисунок 1 - Главная страница Vue-приложения.

2. Структура проекта

После переноса на Vue структура проекта изменилась. Вместо отдельных HTML-страниц и JS-файлов теперь используется структура Vue-проекта.

Основные файлы и папки проекта:

- `index.html` - основной HTML-файл;
- `src/main.js` - точка входа приложения;
- `src/App.vue` - главный компонент;
- `src/router/index.js` - маршруты приложения;
- `src/api/client.js` - настройка axios;
- `src/views` - страницы приложения;
- `src/components` - переиспользуемые компоненты;
- `src/composables` - общая логика;
- `public/icons/sprite.svg` - SVG-спрайт;
- `db.json` - mock база данных.

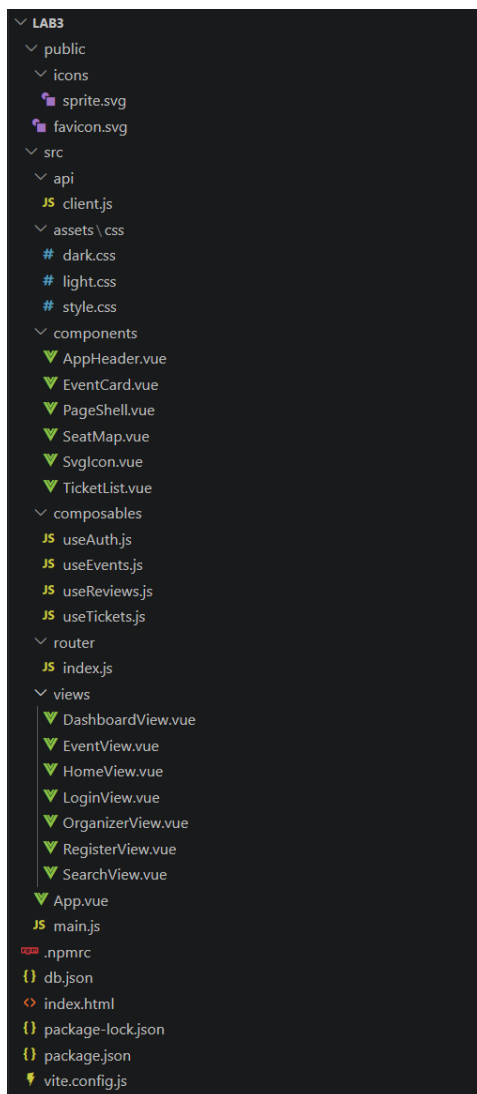


Рисунок 2 - Структура проекта лабораторной работы №3.

3. Подключение роутера

Для маршрутизации был подключён vue-router.

В проекте реализованы маршруты:

- / - главная страница;
- /login - вход;
- /register - регистрация;

- /search - поиск мероприятий;
- /event/:id - страница мероприятия;
- /dashboard - личный кабинет пользователя;
- /organizer - кабинет организатора.

В отличие от предыдущей версии, переходы теперь происходят внутри одного приложения без полной перезагрузки страницы. Это соответствует принципу SPA.

Также в маршрутах используются meta и navigation guards. Они нужны для проверки авторизации и роли пользователя. Например, личный кабинет доступен только обычному пользователю, а кабинет организатора - только организатору.

4. Работа с API через axios

Во второй лабораторной работе проект уже был подключён к mock API. В ЛР3 эта логика была сохранена, но работа с запросами перенесена в Vue-структуру.

Для запросов используется библиотека axios. Настройка находится в файле: src/api/client.js

Через API выполняются:

- вход пользователя;
- регистрация;
- загрузка мероприятий;

- покупка билета;
- возврат билета;
- загрузка отзывов;
- создание мероприятий организатором.

Mock API по-прежнему работает через json-server и использует файл db.json.

```
я\labs\lab3> npm run api
> lab3-from-hw4@1.0.0 api
> json-server db.json -m ./node_modules/json-server-auth --port 3000

\{^_^}/ hi!

Loading db.json
Loading ./node_modules/json-server-auth
Done

Resources
http://localhost:3000/users
http://localhost:3000/events
http://localhost:3000/tickets
http://localhost:3000/returns
http://localhost:3000/reviews

Home
http://localhost:3000

Type s + enter at any time to create a snapshot of the database
```

Рисунок 3 - Запуск mock API.

5. Компонентный подход

Одним из главных отличий ЛР3 стало разделение интерфейса на Vue-компоненты.

В проекте были созданы компоненты:

- AppHeader.vue - шапка сайта;

- EventCard.vue - карточка мероприятия;
- SeatMap.vue - схема зала;
- TicketList.vue - список билетов;
- SvgIcon.vue - компонент для SVG-иконок;
- PageShell.vue - общий контейнер страницы.

Такой подход удобнее, потому что повторяющиеся части интерфейса не нужно копировать на разные страницы. Например, карточка мероприятия используется на странице поиска, а шапка сайта отображается во всём приложении.

6. Использование composable

Для повторяющейся логики была создана папка: `src/composables`

В ней находятся файлы:

- useAuth.js - авторизация, регистрация, выход;
- useEvents.js - работа с мероприятиями;
- useTickets.js - покупка и возврат билетов;
- useReviews.js - работа с отзывами.

Composable позволяют вынести общую логику из компонентов. Например, логика авторизации используется и на странице входа, и в шапке, и при проверке маршрутов.

7. Сохранение SVG-иконок

Так как проект был сделан на основе версии после Д34, в нём были сохранены SVG-иконки.

Файл спрайта находится по пути: `public/icons/sprite.svg`

Для подключения иконок во Vue был создан компонент:

`src/components/SvgIcon.vue`

Иконки используются в шапке, карточках мероприятий и на странице отдельного мероприятия. Таким образом, результат Д34 был сохранён, но адаптирован под Vue.

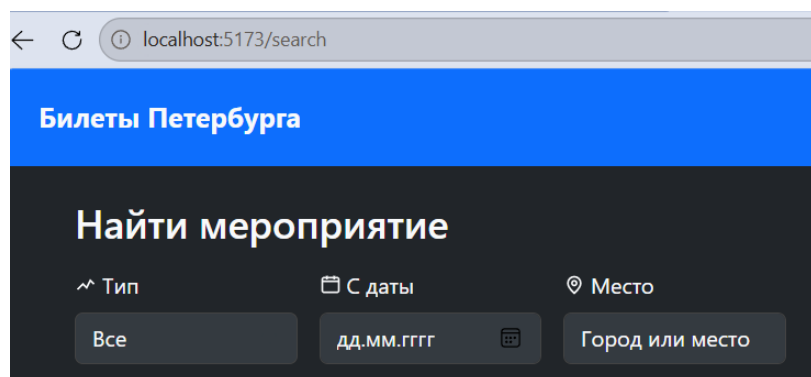


Рисунок 4 - Использование SVG-иконок во Vue-приложении.

8. Основные страницы приложения

В проекте были перенесены все основные страницы.

На странице поиска пользователь может фильтровать мероприятия по типу, дате и месту. Данные загружаются с API, а карточки выводятся через компонент EventCard.

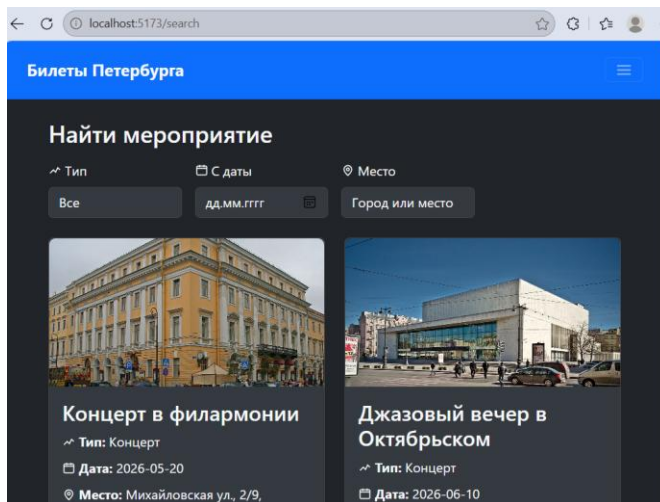


Рисунок 5 - Страница поиска мероприятий.

На странице мероприятия отображаются афиша, название, тип, дата, место, цена, описание, отзывы и схема зала. Для схемы зала используется компонент SeatMap.

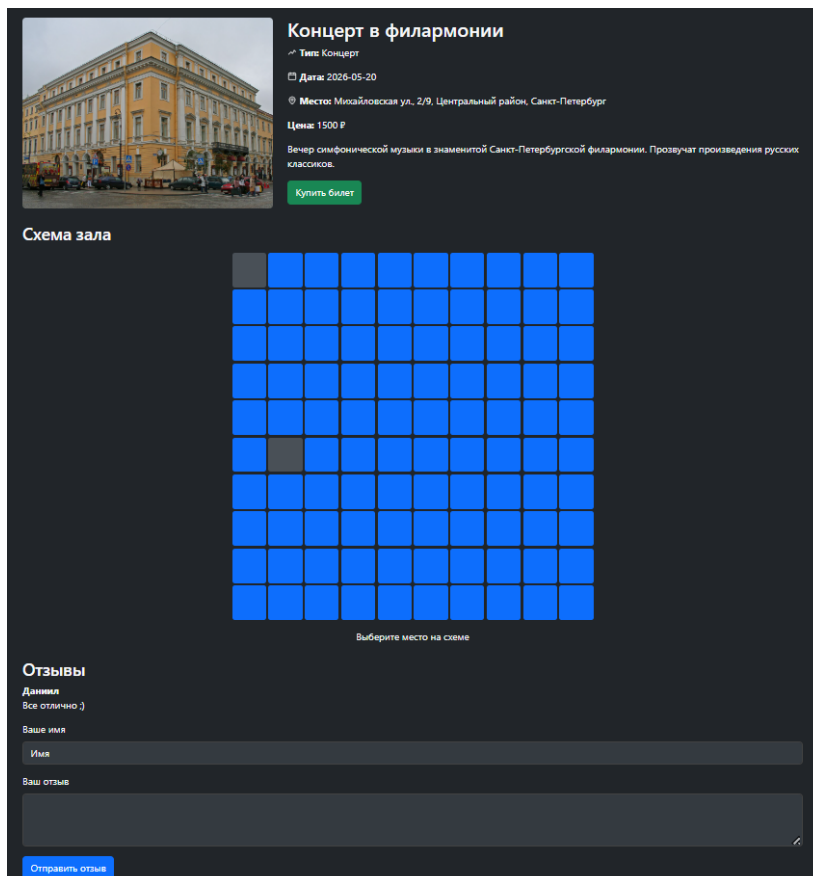


Рисунок 6 - Страница отдельного мероприятия.

В личном кабинете пользователь может посмотреть купленные и возвращённые билеты. Также можно оформить возврат билета.

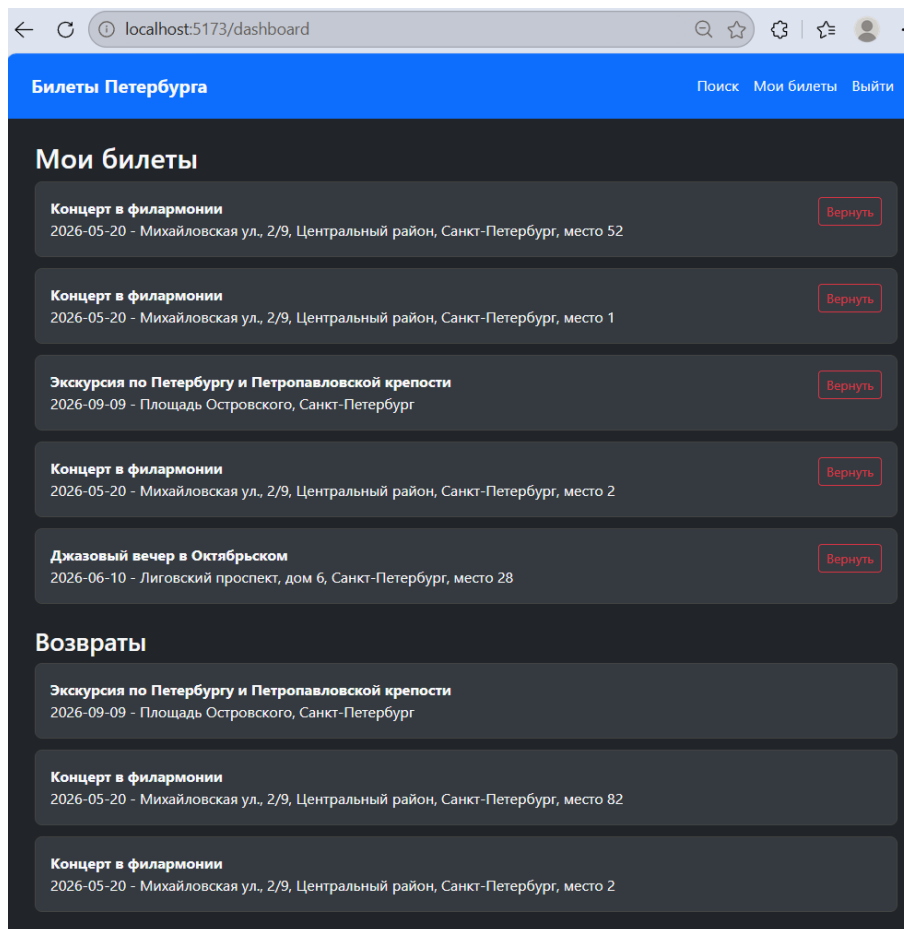


Рисунок 7 - Личный кабинет пользователя.

В кабинете организатора отображаются мероприятия организатора, статистика по билетам и форма создания нового мероприятия.

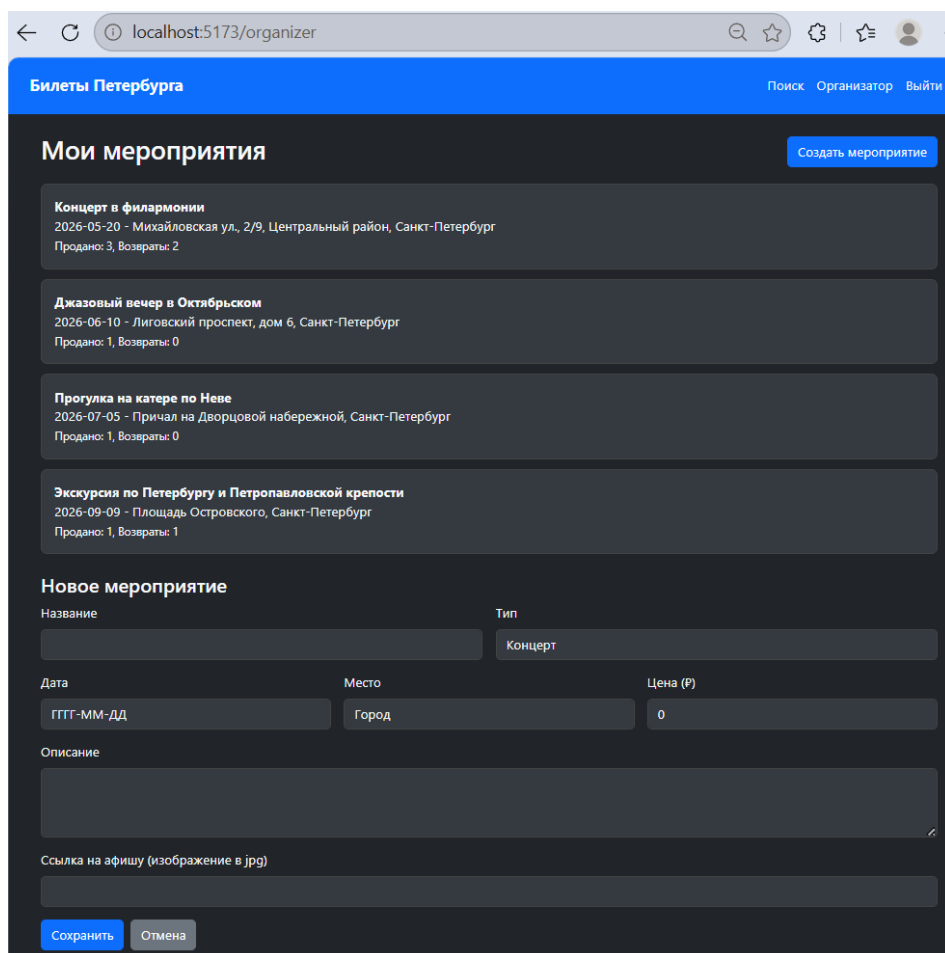


Рисунок 8 - Кабинет организатора.

Что изменилось по сравнению с ЛР1

По сравнению с первой лабораторной работой изменения достаточно большие.

В ЛР1 был создан в основном интерфейс сайта. Страницы были написаны на HTML, CSS, Bootstrap и JavaScript. Они показывали, как должен выглядеть сервис, но большая часть логики была демонстрационной.

В ЛР3 проект стал полноценным SPA-приложением на Vue.JS:

- вместо отдельных HTML-страниц используется одно приложение;
- переходы выполняются через роутер;
- данные загружаются с API;
- интерфейс разбит на компоненты;
- логика вынесена в composable;
- покупка и возврат билетов работают через сервер;
- сохранены SVG-иконки.

То есть по сравнению с ЛР1 проект стал не просто вёрсткой, а работающим клиентским приложением.

Что изменилось по сравнению с ЛР2

По сравнению со второй лабораторной работой внешний вид сайта почти не изменился. Это сделано специально, потому что задача была не переделать дизайн, а мигрировать уже готовое приложение на Vue.JS.

Главные изменения произошли во внутренней структуре проекта.

Во второй лабораторной работе каждая страница была отдельным HTML-файлом:

- index.html;
- login.html;
- register.html;
- search.html;

- event.html;
- dashboard.html;
- organizer.html.

В ЛР3 вместо этого используются Vue-компоненты в папке views.

Также во второй лабораторной работе логика была разбита на обычные JS-файлы, например api.js, auth-service.js, search-page.js, event-page.js. В ЛР3 эта логика перенесена в composable-файлы: useAuth.js, useEvents.js, useTickets.js, useReviews.js.

Основные отличия от ЛР2:

1. Проект стал SPA-приложением.
2. Подключён vue-router.
3. Добавлены meta и navigation guards.
4. Используется компонентный подход.
5. Повторяющаяся логика вынесена в composables.
6. Для API используется axios-клиент.
7. Интерфейс обновляется через реактивность Vue, а не через ручную работу с DOM.
8. SVG-иконки подключаются через отдельный Vue-компонент.

Таким образом, функциональность ЛР2 была сохранена, но приложение стало реализовано по принципам Vue.JS.

Проверка работоспособности

Для проверки проекта были выполнены команды:

```
npm install
```

```
npm run api
```

```
npm run dev
```

Также была проверена сборка:

```
npm run build
```

Приложение запускается без ошибок. В браузере были проверены переходы между страницами, вход пользователя, вход организатора, поиск мероприятий, покупка билета, возврат билета и работа SVG-иконок.

Вывод

В ходе выполнения лабораторной работы №3 я перенесла проект «Платформа для покупки билетов на мероприятия» на фреймворк Vue.JS.

В результате было создано SPA-приложение, в котором сохранены внешний вид и основные функции предыдущих работ. При этом внутренняя структура проекта была изменена: появились Vue-компоненты, маршрутизация, navigation guards, composables-файлы и работа с API через axios.

По сравнению с ЛР1 проект стал не просто интерфейсом, а полноценным приложением. По сравнению с ЛР2 изменилась архитектура: вместо отдельных HTML-страниц и обычных JS-файлов теперь используется Vue SPA.

Считаю, что цель лабораторной работы достигнута, так как приложение было успешно мигрировано на Vue.JS и соответствует требованиям задания.